
Software Testing

Secure Software & Hardware Systems

Today

- Introduction to software testing
 - Testing levels
 - Testing types
 - Test automation
- Testing from a security perspective
 - Test classification
 - Coverage
 - Security testing: First examples
- Recap

Today

- Introduction to software testing
 - Testing levels
 - Testing types
 - Test automation
- Testing from a security perspective
 - Test classification
 - Coverage
 - Security testing: First examples
- Recap

What is Software Testing?

“Testing is the process of analyzing a software item to detect the differences between existing and required conditions (that is, bugs) and to evaluate the features of the software item.”

*“Testing is **the process of analyzing a software item to detect** the differences between existing and required conditions (that is, **bugs**) and to evaluate the features of the software item.”*

Why Software Testing is important

High-level goal: Ensure software behaves as intended

- Software developers are humans
 - We all make mistakes
 - These mistakes can lead to faults/bugs in software
- Testing helps us to identify and fix bugs!
 - Preventing a system from failure
 - Includes identification of security critical bugs (i.e., vulnerabilities)

Terminology: Mistake/Fault/Failure

- Mistake: A human action that produces an incorrect result
 - Synonyms: Error
- Fault: An imperfection or deficiency in a work product where it does not meet its requirements or specifications
 - Synonyms: Defect, Bug
- Failure: An event in which a component or system does not perform a required function within specified limits.



Today

- Introduction to software testing
 - Testing levels
 - Testing types
 - Test automation
- Testing from a security perspective
 - Test classification
 - Coverage
 - Security testing: First examples
- Recap

The four levels of testing



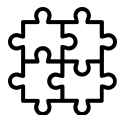
Unit Testing

Target: Individual components



Integration Testing

Target: Modules in combination



System Testing

Target: The fully integrated system



Acceptance Testing

Target: Users of the system

The four levels of testing



Unit Testing

Target: Individual components



Integration Testing

Target: Modules in combination



System Testing

Target: The fully integrated system



Acceptance Testing

Target: Users of the system

- Tests functionality of basic building blocks (e.g, single functions)
- Focuses on components in isolation
- May include stubs or drivers to execute test
- Example:

```
def add(a, b):  
    return a + b  
  
def test():  
    assert add(1,1) == 2
```

The four levels of testing



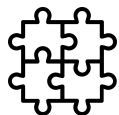
Unit Testing

Target: Individual components



Integration Testing

Target: Modules in combination



System Testing

Target: The fully integrated system



Acceptance Testing

Target: Users of the system

- Tests functionality of building blocks when combined together
- Focuses on interaction between components
- Usually groups (unit tested) modules together following fixed strategies:
 - E.g, top-down, bottom-up, hybrid

- Example:

```
def test():  
    assert sub(add(1,1),2) == 0
```

The four levels of testing



Unit Testing

Target: Individual components

- Tests functionality of fully integrated system
- Focuses on interaction between all components and the correctness of system as a whole
- May be guided by requirement specifications



Integration Testing

Target: Modules in combination



System Testing

Target: The fully integrated system

- Example:

```
$ calculator.py "2+2-1"
```



Acceptance Testing

Target: Users of the system

The four levels of testing



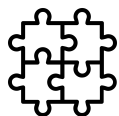
Unit Testing

Target: Individual components



Integration Testing

Target: Modules in combination



System Testing

Target: The fully integrated system



Acceptance Testing

Target: Users of the system

- Tests whether system fulfills acceptance criteria (e.g., requirement/specification)
- May involve (end-)users of the system
- Includes well-known testing types:
 - Alpha testing, beta testing

- Example:



Where is
division?

```
$ calculator.py 2+2/1  
ERROR: No such operand "/"
```

Today

- Introduction to software testing
 - Testing levels
 - Testing types
 - Test automation
- Testing from a security perspective
 - Test classification
 - Coverage
 - Security testing: First examples
- Recap

Test types

A test type is:

“A group of test activities based on specific test objectives aimed at specific characteristics of a component or system.”

Test types

A test type is:

“A group of test activities based on **specific test objectives** aimed at **specific characteristics** of a component or system.”

Test types

- Focus on specific objectives and characteristics
 - While test levels focus on which parts of the system are tested
 - However: each level can also be seen as a test type
- Other example test types:
 - Regression tests, smoke tests, conformance tests, security tests

T

Regression Tests

- Tests to ensure that introduced changes do not cause new defects
 - These defects would be “regressions”
- Most common method: re-run all tests after change
 - If not possible: selection of specific regression tests or prioritization of specific test cases
- Does not assess correctness of new parts of the software!



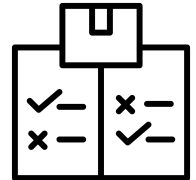
Smoke Tests

- Non-exhaustive tests to assess main/critical functionality of the software
 - Usually early on in testing/development process
- Used to determine if further testing should proceed
 - If smoke test fails, additional tests make likely no sense!
- Examples:
 - Does the software start?
 - Can a user provide input?



Conformance tests

- Tests to ensure that software behaves according to their respective standard/specification
 - Test cases itself may be standardized as well
 - May be part of certification process
 - Especially relevant for safety-critical systems
- Example applications:
 - Compilers
 - Cellular Basebands



Security Tests

- Specialized tests to find security critical defects in the target software
 - May be carried out by 3rd-parties, rather than development team
- Can focus on different types of security related bugs, for instance:
 - Logic bugs (e.g., authentication bypass, API misuse, etc.)
 - Configuration errors (e.g., use of weak ciphers, default credentials, etc)
 - Memory corruptions (e.g., stack-based buffer overflows, use-after-free, etc)
- Examples:
 - Source code audits, fuzz testing, symbolic execution
- More later

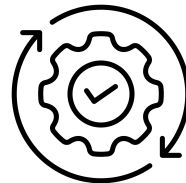


Today

- Introduction to software testing
 - Testing levels
 - Testing types
 - Test automation
- Testing from a security perspective
 - Test classification
 - Coverage
 - Security testing: First Examples
- Recap

Test Automation

- Automating test execution and result analysis is highly beneficial
 - We don't want to execute all our tests by hand!
- Most languages provide test automation frameworks, e.g.:
 - Python: Pytest, Pyunit, behave
 - Rust: built-in testing (`#[test]`)
 - C: Check, Unity Test, custom Make files



Continuous integration and deployment (CI/CD)

- Commonly used in modern software engineering
 - As part of a CI/CD pipeline
- Core ideas:
 - Merge changes *frequently* into a single mainline
 - Before merge, ensure functionality via automated tests
- Examples:
 - Github actions, gitlab runners, travis CI

```
name: CI example

on: [push]

jobs:
  example-job:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v2
      - name: run tests
        run: ./run_tests.sh
```

Github action example

Today

- Introduction to software testing
 - Testing levels
 - Testing types
 - Test automation
- **Testing from a security perspective**
 - Test classification
 - Coverage
 - Security testing: First examples
- Recap

The “Box Model”

Testing can be classified based on the information provided to the test/tester:



Black-box

- No information about the internal structure
- Tester observes input and output



Grey-box

- Partial information about the internal structure
- Tester can create informed tests



White-box

- Full information about the internal structure
- Tester can leverage source code

Static vs. Dynamic Testing

Static

- Also called “off-line” testing
- Testing without execution
- *Reasons* about the code
- May create false-positives

Dynamic

- Also called “on-line” testing
- Testing by executing the target
- Observes the code’s *behavior*
- May not cover full program

Today

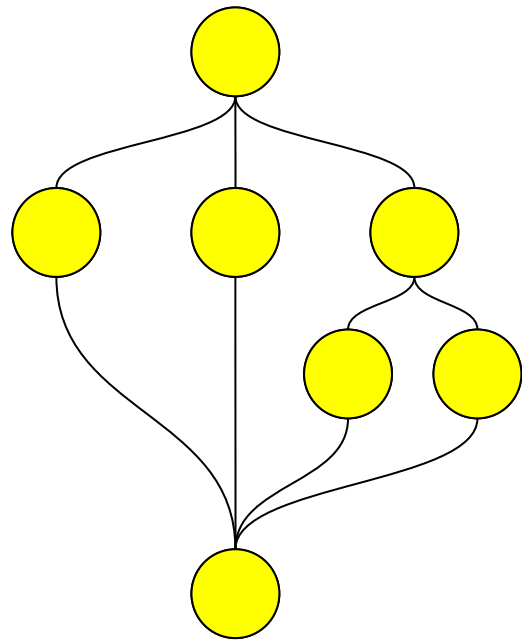
- Introduction to software testing
 - Testing levels
 - Testing types
 - Test automation
- **Testing from a security perspective**
 - Test classification
 - Coverage
 - Security testing: First examples
- Recap

Test Coverage

- Metrics to understand how complete given tests are
 - Provides insights which parts of a software are tested
- Multiple types, e.g.:
 - Requirement coverage (*which of the requirements are tested?*)
 - Compatibility coverage (*which hardware/software targets is the target tested with?*)
 - Code coverage (*which parts of the code are tested?*)
- We will look more into code coverage
 - Useful metric for security testing
 - More code coverage == more parts of the system tested

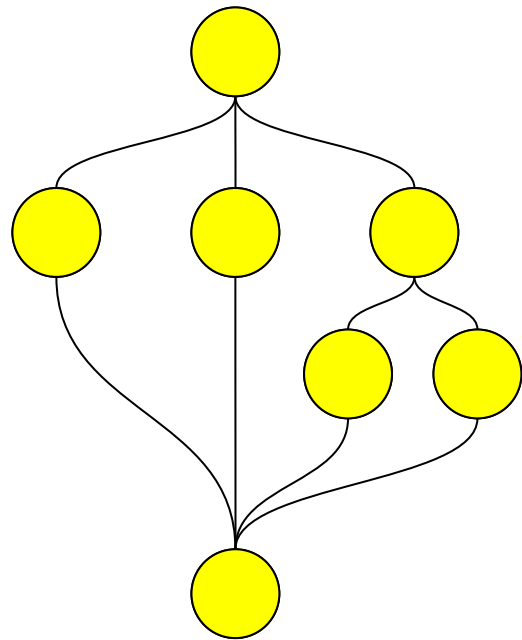
Code Coverage: Example

```
void my_function(char in1, char in2) {  
    switch (in1) {  
        case 'A': handle_A(); break;  
        case 'B':  
            if (in2 == 'S') {  
                handle_B_special(); break;  
            } else {  
                handle_B_default(); break;  
            }  
        default: housekeeping();  
    }  
    return;  
}
```



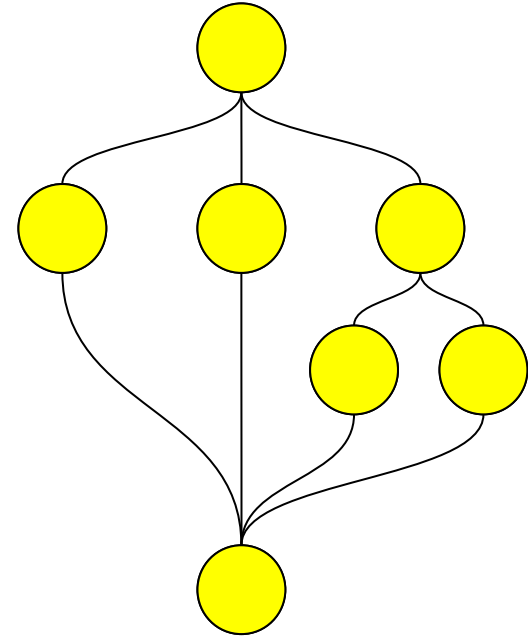
Code Coverage: Ideal scenario

```
void my_function(char in1, char in2) {  
    switch (in1) {  
        case 'A': handle_A(); break;  
        case 'B':  
            if (in2 == 'S') {  
                handle_B_special(); break;  
            } else {  
                handle_B_default(); break;  
            }  
        default: housekeeping();  
    }  
    return;  
}
```



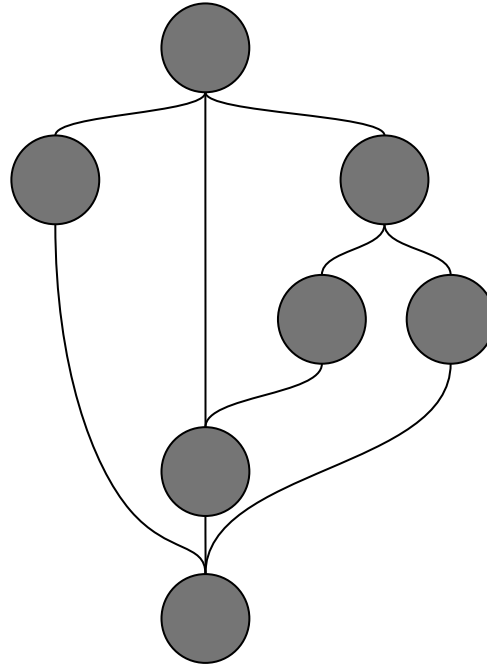
Code Coverage Types

- Function coverage
 - Which parts of the *software* are executed?
- Line coverage
 - Which parts of the *source code* are executed?
- Block coverage
 - Which parts of the *machine code* are executed?
- Edge coverage
 - Which parts of the control flow graph are executed?



Quiz

```
void my_function(char in1, char in2) {  
    switch (in1) {  
        case 'A': handle_A(); break;  
        case 'B':  
            if (in2 == 'S') {  
                handle_B_special(); NO BREAK!  
            } else {  
                handle_B_default(); break;  
            }  
        default: housekeeping();  
    }  
    return;  
}
```

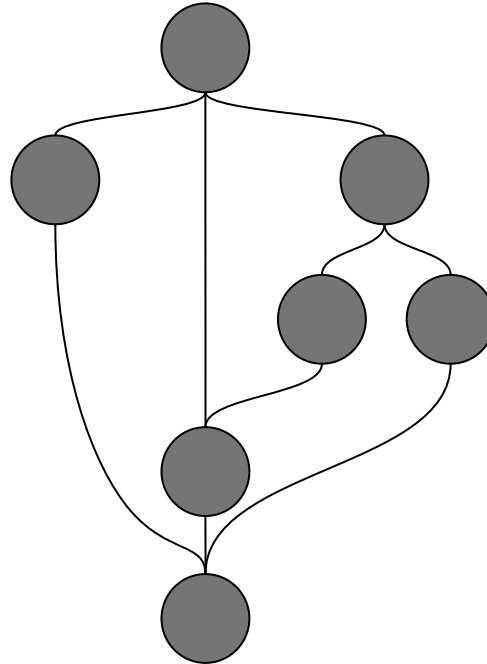


How many test cases (i.e., combinations of inputs) are required to achieve full:

- 1) Function coverage?
- 2) Line coverage?
- 3) Block coverage?
- 4) Edge coverage?

Quiz

```
void my_function(char in1, char in2) {  
    switch (in1) {  
        case 'A': handle_A(); break;  
        case 'B':  
            if (in2 == 'S') {  
                handle_B_special();  
            } else {  
                handle_B_default(); break;  
            }  
        default: housekeeping();  
    }  
    return;  
}
```



How many test cases (i.e., combinations of inputs) are required to achieve full:

- 1) Function coverage? 3
- 2) Line coverage? 3
- 3) Block coverage? 3
- 4) Edge coverage? 4

Code coverage: Caveats

- Writing complete tests is not trivial!
- Edge coverage most complete:
 - Function coverage may miss large parts of functionality
 - Line coverage assumes functionality isolated in individual lines
 - Block coverage does not track all transition
- Code coverage may not give insights about data-dependent faults:
 - E.g., consider off-by-one errors in loops or the arguments to a `strcpy` call

Today

- Introduction to software testing
 - Testing levels
 - Testing types
 - Test automation
- Testing from a security perspective
 - Test classification
 - Coverage
 - Security testing: First examples
- Recap

Static Code Analysis

- Most common scenario: source code analysis
 - However, techniques for static binary code analysis exist as well!
- Example: Detecting vulnerable calls to APIs
 - Could be as simple as ``$ grep strcpy``
 - Or using advanced tooling such as CodeQL [1]
- Scales well to full programs
 - But may report many false positives

```
import python

from Function f
where count(f.getAnArg()) > 7
select f
```

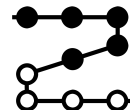
CodeQL Example [2]

[1] <https://codeql.github.com/>

[2] <https://codeql.github.com/docs/writing-codeql-queries/introduction-to-ql/>

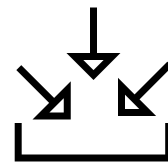
Taint Analysis

- Testing techniques which can be implemented both dynamically or statically
- Aims to identify flow of user input throughout the program
 - And whether it can has security implications
- Core idea:
 - Tagging user input and tracking (“tainting”) its use through the program
 - Check whether tainted values ever reach vulnerable functions (“sinks”) without prior sanitization



Fuzzing

- Executing the target program repeatedly with (pseudo-)random input
 - Mutate existing or generate new input for each run
- Monitor program behaviour for crashes
 - Crashes demonstrate shortcomings (failures) of the target software
- Fuzzing has been shown **very** effective for finding security vulnerabilities
 - We will have a deep dive next lecture
- Scales well to full programs
 - But may miss substantial parts of the target



Symbolic Execution

- Instead of testing/executing the target with *concrete* inputs “execute” it with *symbolic* values
 - Goal: “Exploring” all states of a program
- On each branch, collect constraints on inputs
 - If test case to reach a certain state is required, invoke a solver
- Scalability issues:
 - State/Path explosion
 - Constraint solving

Quiz

```
void my_function(char in1, char in2) {  
  switch (in1) {  
    case 'A': handle_A(); break;  
    case 'B':  
      if (in2 == 'S') {  
        handle_B_special();  
      } else {  
        handle_B_default(); break;  
      }  
    default: housekeeping();  
  }  
  return;  
}
```

How many test cases (i.e., combinations of inputs) are required to achieve full:

1) Function coverage?	1
2) Line coverage?	3
3) Block coverage?	3
4) Edge coverage?	4

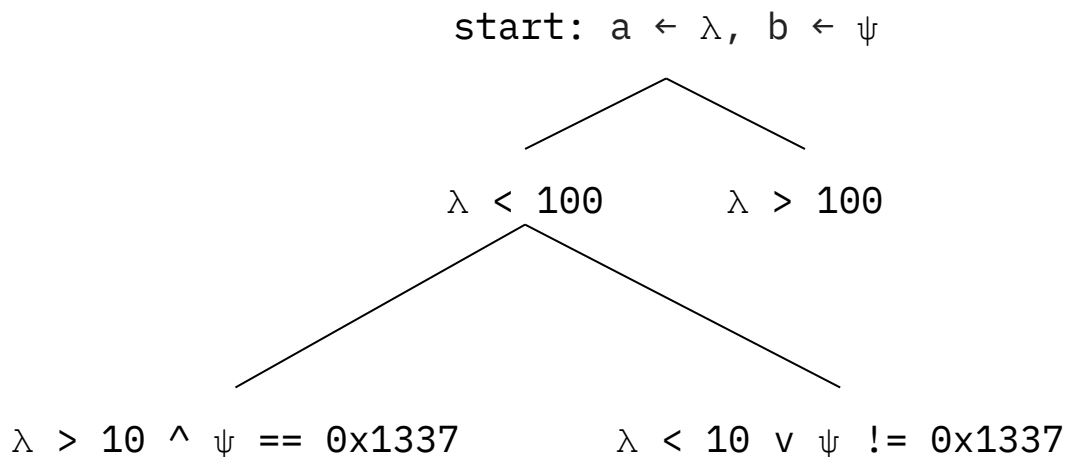
Example test cases: (1) AX - (2) AX, BX, BS - (3) AX, BX, BS - (4) AX, BX, BS, CX

*Simplified example: interpreting
inputs to program state
relations*

Symbolic Execution: Example

```
→ void fn(int a, int b) {  
  → if (a < 100) {  
    → if (a > 10 && b == 0x1337) {  
      → bug();  
    } else {  
      puts("All good!");  
    }  
  } else {  
    puts("Nothing to see here");  
  }  
}
```

During symbolic execution, we collect constraints:



Taking the constraints together, we can solve for bug, e.g.: $a = 11, b = 0x1337$

Today

- Introduction to software testing
 - Testing levels
 - Testing types
 - Test automation
- Testing from a security perspective
 - Test classification
 - Coverage
 - Security testing showcase
- Recap

Recap

We looked into:

- The basics of software testing
 - What are tests and why do we need them?
- Relevant concepts for security testing
 - Which parts of software testing aid security analysis?
- Example security testing techniques
 - How does testing help us as security analysts?